

Team Root Matrix: Mary Ann Mastri, Cesar Noriega, Luiz Paludo, Nichelle Graf
L04 CNNs Using MNIST Dataset – Team Reflections
ITAI 2376 – Deep Learning in AI
Professor Viswanatha Rao

L04 - CNNs Using MNIST Dataset

Team Reflections

Mary Ann:

I experimented with different CNN architecture for the MNIST dataset and learned more than I expected. I changed the number of filters, pooling layers, batch sizes and epochs. This lab taught me that I can change many of the inputs that will affect the output of the model.

I learned how important the Conv2D layers are. They actually see the image, and so I tested increasing the number of filters to 128. It gave the model the ability to learn more features, but it also made it more sensitive to other things like pooling and how long it was trained. I didn't bother changing the kernel size because 3x3 is already the standard for small images and changing it would have just been an exercise in suboptimal results.

I focused mostly on changing the number of epochs to increase the length of learning, and batch size to increase or decrease the images it will learn from. The smaller epoch sizes of 15 all seemed to perform the same regardless of the batch size. When I increased the epoch to 25, then the larger batch sizes had the highest accuracy. But the test loss also increased – so it was learning more aggressively and also more confident in its mistakes.

This lab let me experiment and understand how small changes in the architecture can affect training and it gave me the opportunity to become more confident in reading the model outputs and interpreting what they mean.

Cesar:

In this lab, I understood better how a CNN works to recognize images. I learned that Conv2D looks for patterns such as lines, edges, curves, and shapes, while MaxPooling reduces the size of the information while keeping the important patterns. I also understood that one-hot encoding changes the number from the image into a vector with 10 categories, and Flatten converts the information into a vector so it can go to the Dense layer. The Dropout layer helps prevent overfitting. At first, it was a little difficult for me to understand how all these layers work together, but by running each part in Google Colab, I could understand the process better. I also learned that the model processes the images in groups of 128 and uses 15 epochs for training. The result surprised me because we got 99.08% test accuracy with a test loss of 0.0249. This lab helped me understand in a more practical way how a CNN processes an image and finally identifies the number.

Luiz:

For this lab, we built a CNN model in Keras to classify handwritten digits from the MNIST dataset. It was a good exercise because I could see how the whole process works step by step. First, we had to prepare the data. We divided the pixels to normalize them between 0 and 1, then reshaped the images

and used one-hot encoding for the labels. This step is important so the network can read the images correctly. My main focus on the project was the training setup and testing. Based on my experience from a past project, I helped pick the batch size of 128 and 15 epochs. During training, I watched the validation accuracy and loss to make sure the model was learning without overfitting. In the end, our model reached 99.08% test accuracy and a test loss of 0.0249. I am happy with these results because it shows the Conv2D, Dropout, and training settings worked together well to make good predictions.

Nichelle:

This lab helped me understand more clearly how the different parts of a convolutional neural network work together to classify images. I had seen concepts such as convolutional layers, pooling, dropout, and one-hot encoding before, but working through the MNIST notebook made the process more concrete. I could see how the Conv2D layers detect visual features, how MaxPooling reduces the amount of information being processed while preserving important patterns, and how the Flatten and Dense layers convert those learned features into a final classification.

One part that was especially useful was experimenting with the model instead of only running the default code. I changed the second Conv2D layer from 64 filters to 128 filters, then later changed the training parameters from a batch size of 128 and 15 epochs to a batch size of 64 and 10 epochs. The experimental training run reached 99.10% training accuracy and 99.13% validation accuracy, with a highest validation accuracy of 99.30%. The final test accuracy was 99.19% with a test loss of 0.0253. What stood out to me was that reducing the number of epochs did not meaningfully reduce performance, which showed me that more training is not automatically better.

One challenge was understanding the difference between changing the model architecture and actually evaluating the effect of that change. Simply changing a parameter in the model definition does not produce evidence by itself. The results only become meaningful after the model is trained and evaluated. That distinction helped me better understand how experiments in deep learning should be structured.

Overall, this lab gave me a more practical understanding of CNNs and how model architecture and training parameters affect performance. I also became more comfortable reading training and validation results instead of looking only at the final accuracy. I would like to explore additional changes in future models, such as different kernel sizes, dropout rates, and additional convolutional layers, to see how those choices affect both accuracy and efficiency.